DEEP REINFORCEMENT LEARNING
365.382 (SS26)

Value Function Approximation

Sohvi Luukkonen
LIT AI LAB
Institute for Machine Learning

JKU
JOHANNES KEPLER
UNIVERSITY LINZ

Resources

- Reinforcement Learning: An Introduction [Chapters 6, 9 to 11]
- DeepMind x UCL | RL Lecture Series 2021 [Lectures 6, 7, 12 and 13]

Today's Outline

- Recap: Tabular RL methods
- Q-Learning
- Value Function Approximation
- Deep Q-Networks (DQN)
- *Introduction DQN Assignment*

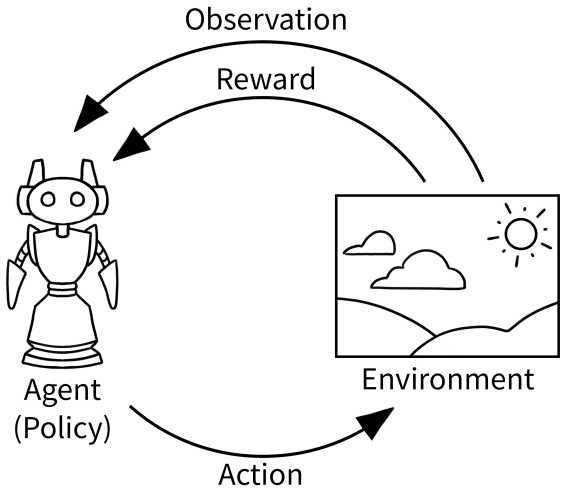
Recap: Tabular RL Methods

What is Reinforcement Learning?

- Reinforcement learning (RL) is learning to make decisions from interactions.
- The objective is not immediate reward, but long-term return.
- Unlike supervised learning, RL has no labeled optimal action for each state.
- Core challenge: exploration vs. exploitation.

Key question: How can we learn a policy that maximizes expected cumulative reward?

The Interaction Loop



- Agent chooses action A_t in state S_t
- Environment returns reward R_{t+1} and next state S_{t+1}
- Repeat until terminal state

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Goal: maximize expected return $\mathbb{E}[G_t]$.

Bellman Expectation Equations

Theorem (Bellman Expectation Equations)

Given and MDP, $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, for any policy π, the value functions obey the following expectation equations:

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi \left[r + \gamma v_\pi(s') | S = s \right] \\
&= \sum_a \pi(a|s) \Big[r(s,a) + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \Big] \\
q_\pi(s,a) &= \mathbb{E}_\pi \left[r + \gamma q_\pi(s', a') | S = s, A = a \right] \\
&= r(s,a) + \gamma \sum_{s'} p(s'|s,a) \sum_{a'} \pi(a'|s') q_\pi(s', a')
\end{aligned}
$$

Used for prediction (policy evaluation).

Bellman Optimality Equations

Theorem (Bellman Optimality Equations)

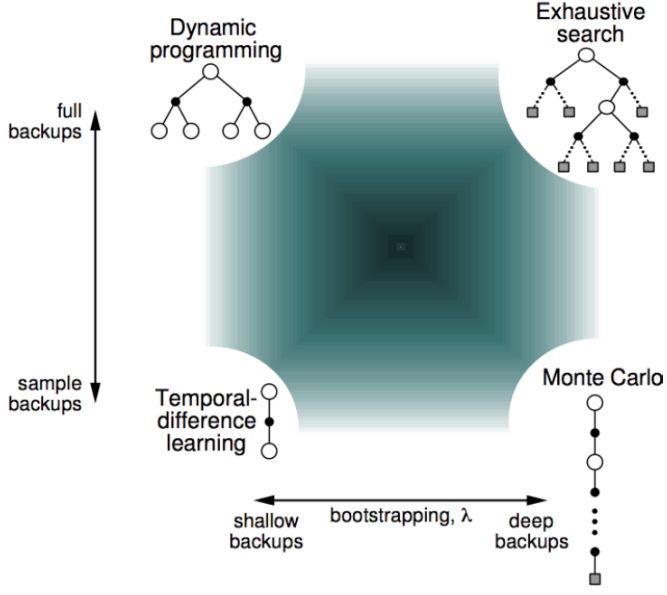
Given and MDP, $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, the optimal value functions obey the following expectation equations:

$$v^*(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} p(s'|s,a) v^* s' \right]$$

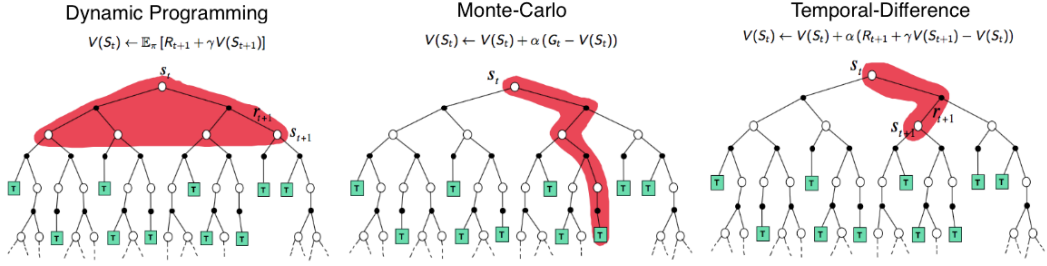
$$q^*(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|a,s) \max_{a'} q^*(s',a')$$

Used for control (finding optimal policy).

Approaches to Solving Bellman Equations



Bootstrapping and Sampling



Bootstrapping

update involves an estimate
based on previous estimates
(DP & TD)

Sampling

update involves samples
of an expectation
(MC & TD)

Dynamic Programming Algorithms

DP algorithms iteratively apply Bellman backups to compute value functions:

$$v_{k+1}(s) = \mathbb{E}\Big[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t \sim \pi(S_t)\Big] \quad \text{(policy evaluation)}$$

$$v_{k+1}(s) = \max_a \mathbb{E}\Big[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a\Big] \quad \text{(value iteration)}$$

$$q_{k+1}(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma q_k(S_{t+1}, A_{t+1}) | S_t = s, A_t = a\Big] \quad \text{(policy evaluation)}$$

$$q_{k+1}(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_k(S_{t+1}, a') | S_t = s, A_t = a\Big] \quad \text{(value iteration)}$$

TD Algorithms

Analogously, model-free TD algorithms iteratively apply sampled Bellman backups to compute value functions:

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t \Big[R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t) \Big] \qquad \text{(TD)}$$

No trivial analogous for value iteration for state values.

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t \Big[R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t) \Big] \qquad \text{(SARSA)}$$

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t \Big[R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t) \Big] \qquad \text{(Q-Learning)}$$

On and Off-Policy Learning

- On-policy learning: Learn about behavior policy π from experiences generated by π
- Off-policy learning: Learn about target policy π from experiences generated by a different behavior policy μ
  - Learn from observing humans or other agents (e.g., from logged data)
  - Re-use experience from old policies (e.g., from your own past experience)
    - This is the key for replay-based deep RL later.
  - Learn about multiple policies while following one policy
  - Learn about greedy policy while following exploratory policy.

Q-Learning

Off-Policy TD Control: Q-Learning

Q-Learning update rule:

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \Big[R_{t+1} + \gamma \max_{a'} Q_{\text{old}}(S_{t+1}, a') - Q_{\text{old}}(S_t, A_t) \Big]$$

is a sample-based approximation of the Bellman optimality equation that estimates the value of the greedy policy.

Theorem

Tabular Q-learning control converges to the optimal action-value function, $q \to q^*$, as long as we take each action in each state infinitely often and the step sizes α_t decrease appropriately (i.e., Robbins-Monro conditions: $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$).

No need for greedy behavior!

SARSA vs Q-Learning

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

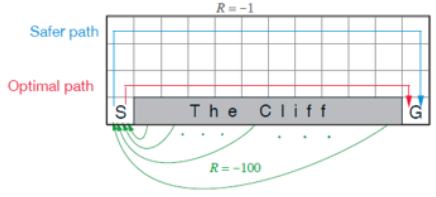
Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal



Expected SARSA

- SARSA: next action A_{t+1} is sampled from the behavior policy $A_{t+1} \sim \mu(\cdot|S_{t+1})$.
- Instead, we can take the expectation under a target policy π and update $Q(S_t, A_t)$ toward:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)\Big]$$

- More expensive than SARSA, but typically lower variance and more stable.
- If $\pi = \mu$: on-policy, if $\pi \neq \mu$: off-policy.
- If π is greedy, this reduces to Q-learning.

TD Control Comparison

Property	SARSA	Q-Learning	Expected SARSA
Policy type	On-policy	Off-policy	On/Off-policy
Target	$R + \gamma Q(S', A')$	$R + \gamma \max_{a'} Q(S', a')$	$R + \gamma \mathbb{E}_{a' \sim \pi}[Q(S', a')]$
Behavior	Safe	Aggressive	Smooth
Convergence	GLIE + RM	RM	GLIE + RM

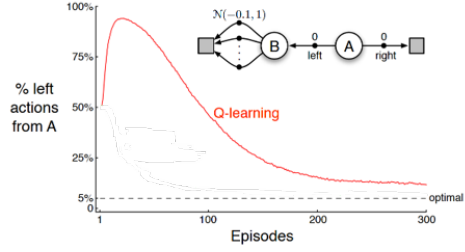
Maximization Bias in Q-Learning

Q-Learning's max operator uses the same values to select and to evaluate:

$$\max_{a'} Q_t(S_{t+1}, a') = Q_t\Big(S_{t+1}, \arg\max_{a'} Q_t(S_{t+1}, a')\Big).$$

... but values are noisy estimates, so we can get overestimation bias:

- more likely to select overestimated values
- less likely to select underestimated values



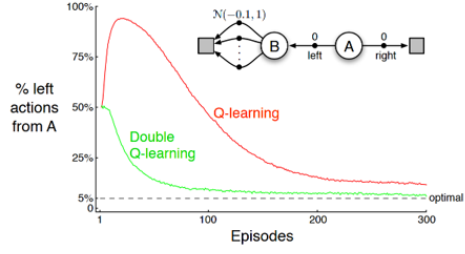
Double Q-Learning

Store two estimates Q^A and Q^B and use one to select and the other to evaluate:

$$Q^A(S_t, A_t) \leftarrow Q^A(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q^B(S_{t+1}, \operatorname*{argmax}_a Q^A(S_{t+1}, a)) - Q^A(S_t, A_t)\big],$$

$$Q^B(S_t, A_t) \leftarrow Q^B(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q^A(S_{t+1}, \operatorname*{argmax}_a Q^B(S_{t+1}, a)) - Q^B(S_t, A_t)\big].$$

- At each t, randomly update either Q^A or Q^B.
- Can use both to act (e.g., greedy w.r.t. $(Q^A + Q^B)/2$).
- Decouples selection and evaluation.
- Reduces overestimation bias.



Idea of Double Q-learning can be generalize to other updates (e.g. ϵ-greedy SARSA).

Value Function Approximation

Large-Scale Reinforcement Learning

Reinforcement learning can be used to solve large problems, e.g.

- Backgammon: 10^{20} states
- Go: 10^{170} states
- Drone navigation: continuous state space
- Robotic manipulation: real world

Function Approximation and Deep RL

- The policy, value functions, and models are all functions
- Learn these from experience
- If there are too many states, we need to approximate these functions with a parameterized function approximator (e.g., linear function, neural network)
- If neural networks are used, we call it deep reinforcement learning

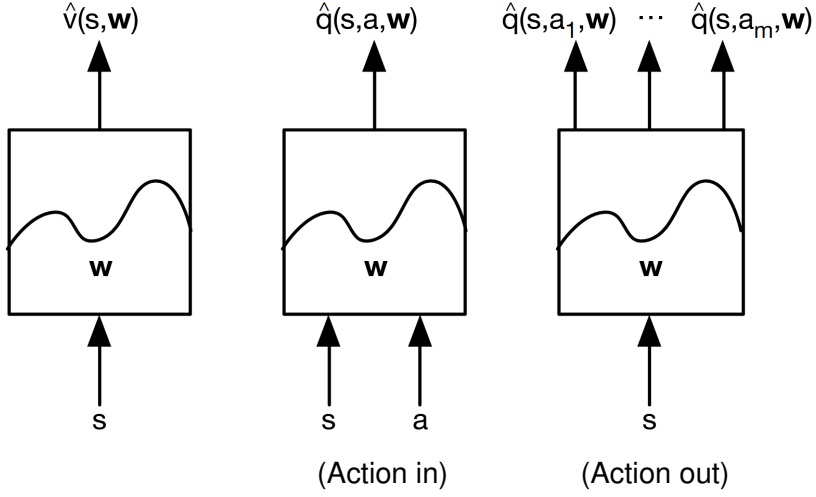
Value Function Approximation

- So far, we have only considered tabular methods that store a separate value for each state (or state-action pair).
- This is not feasible for large state spaces:
  - Memory constraints: cannot store a value for each state.
  - Sample efficiency: cannot visit each state enough times to learn accurate values.
- Instead, we can estimate value functions with function approximation:

$$v_{\mathbf{w}}(s) \approx v_\pi(s) \qquad (\text{or } v^*(s))$$
$$q_{\mathbf{w}}(s,a) \approx q_\pi(s,a) \qquad (\text{or } q^*(s,a))$$

  - Update parameter $\mathbf{w}$ (e.g., using MC or TD learning)
  - Generalize to unseen states

Types of Value Function Approximation



Classes of Function Approximators

- Linear function approximation: $v_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$, where $\mathbf{x}(s)$ is a feature vector.
  - Tabular: a separate parameter for each state (or state-action pair) can be seen as linear function approximation with one-hot encoding features.
  - State aggregation: partitioning states into clusters can be seen as linear function approximation with binary features indicating cluster membership.
- Nonlinear function approximation:
  - Non-differentiable: e.g., decision trees, nearest neighbors, etc.
  - Differentiable: e.g., neural networks, kernel methods, etc.

Classes of Function Approximators

In principle, any function approximator can be used, but RL has specific properties:

- Non-i.d.d. data – successive time-steps are correlated
- Agent's policy affects the data it receives
- Non-stationary regression targets – target values change as the agent learns
  - ...because of changing policies
  - ...because of bootstrapping
  - ...because of non-stationary dynamics (e.g., other learning agents)
  - ...because the world is large (never quite in the same state)

Classes of Function Approximators

Which function approximation should you choose? This depends on your goals.

- Tabular: good theory but does not scale/generalize
- Linear: reasonably good theory, but requires good features
- Non-linear: less well-understood, but scales well
- Flexible, and less reliant on picking good features first (e.g., by hand)
- (Deep) neural nets often perform quite well, and remain a popular choice in practice, but they are not the only option.

Value Function Approx. By Stochastic Gradient Descent

- Approximate true value function $v_\pi(s)$ with a parameterized function $v_{\mathbf{w}}(s)$.
- Minimize mean-squared error between $v_{\mathbf{w}}(s)$ and target $v_\pi(s)$:

$$J(\mathbf{w}) = \mathbb{E}_\pi\Big[\big(\underbrace{v_\pi(S)}_{\text{oracle}} - v_{\mathbf{w}}(S)\big)^2\Big]$$

- Gradient descent finds a local minimum:

$$\begin{aligned}\Delta\mathbf{w} &= -\frac{1}{2}\alpha\nabla_{\mathbf{w}} J(\mathbf{w}) = -\frac{1}{2}\alpha\nabla_{\mathbf{w}}\mathbb{E}_\pi\Big[\big(v_\pi(S) - v_{\mathbf{w}}(S)\big)^2\Big] \\ &= \alpha\mathbb{E}_\pi\Big[\big(v_\pi(S) - v_{\mathbf{w}}(S)\big)\Big]\nabla_{\mathbf{w}} v_{\mathbf{w}}(S)\end{aligned}$$

- Stochastic gradient descent samples the gradient:

$$\Delta\mathbf{w} = \alpha\big(v_\pi(S) - v_{\mathbf{w}}(S)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(S)$$

Feature Vectors

- Represent state by a feature vector

$$\mathbf{x}(s) = \begin{bmatrix} x_1(s) \\ \vdots \\ x_d(s) \end{bmatrix}$$

- $\mathbf{x} : \mathcal{S} \to \mathrm{R}^n$ is a fixed mapping from states to features.
- For example:
  - Distance of robot from landmarks
  - Trends in the stock market
  - Piece and pawn configurations in chess

Linear Function Approximation

- Approximate value function as a linear combination of features:

$$v_{\mathbf{w}}(s) = \mathbf{x}(s)^\top \mathbf{w} = \sum_{i=1}^{n} w_i x_i(s)$$

- Objective function is quadratic in parameters $\mathbf{w}$, so it has a unique global minimum:

$$J(\mathbf{w}) = \mathbb{E}_\pi \Big[\big(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w} \big)^2 \Big]$$

- Update rule for stochastic gradient descent:

$$\Delta \mathbf{w} = \underbrace{\alpha \big(v_\pi(S) - v_{\mathbf{w}}(S) \big) \mathbf{x}(S)}_{\text{step size} \times \text{prediction error} \times \text{feature value}}$$

Tabular Lookup Features

- Tabular lookup can be seen as linear function approximation with one-hot encoding features
- Using one-hot encoding, we can represent any function exactly (i.e., no generalization)

$$\mathbf{x}^{table}(s) = \begin{bmatrix} \mathbb{1}_{s=s_1} \\ \mathbb{1}_{s=s_2} \\ \vdots \\ \mathbb{1}_{s=s_{|\mathcal{S}|}} \end{bmatrix}$$

- Parameters $\mathbf{w}$ directly represent the value function:

$$v_{\mathbf{w}}(s) = \mathbf{x}^{table}(s)^\top \mathbf{w} = \begin{bmatrix} \mathbb{1}_{s=s_1} \\ \mathbb{1}_{s=s_2} \\ \vdots \\ \mathbb{1}_{s=s_{|\mathcal{S}|}} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{w}_1 \\ \mathbf{w}_2 \\ \vdots \\ \mathbf{w}_{|\mathcal{S}|} \end{bmatrix}$$

Incremental Prediction Algorithms

- We don't have access to the true value $v_\pi(S)$
- We substitute $v_\pi(S)$ for a target:

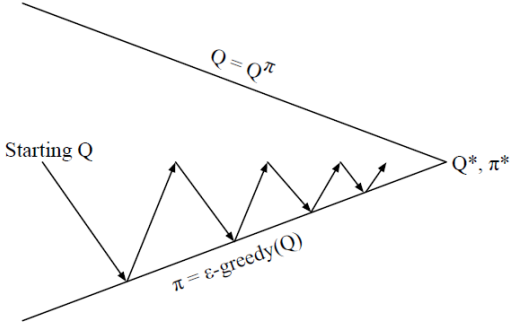
$$\Delta \mathbf{w}_t = \alpha\big(G_t - v_{\mathbf{w}}(S)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(S) \qquad \text{(MC)}$$

$$\Delta \mathbf{w}_t = \alpha\big(R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(S) \qquad \text{(TD(0))}$$

$$\Delta \mathbf{w}_t = \alpha\big(G_t^\lambda - v_{\mathbf{w}}(S)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(S) \qquad \text{(TD}(\lambda)\text{)}$$

$$G_t^\lambda = R_{t+1} + \gamma\big((1-\lambda)v_{\mathbf{w}}(S_{t+1}) + \lambda G_{t+1}^\lambda\big)$$

Control with Function Approximation



- Policy evaluation: *Approximate* policy evaluation, $q_{\mathbf{w}} \approx q_\pi$
- Policy improvement: ϵ-greedy policy improvement, $\pi(s) = \epsilon\text{-greedy}(q_{\mathbf{w}}(s, \cdot))$

Action-Value Function Approximation

- Approximate true action-value function $q_\pi(s, a)$ with a parameterized function $q_{\mathbf{w}}(s, a)$.
- Objective: minimize mean-squared error between $q_{\mathbf{w}}(s, a)$ and target $q_\pi(s, a)$:

$$J(\mathbf{w}) = \mathbb{E}_\pi\Big[\big(\underbrace{q_\pi(S, A)}_{\text{oracle}} - q_{\mathbf{w}}(S, A)\big)^2\Big]$$

- Stochastic gradient descent finds a local minimum:

$$\begin{aligned}\Delta\mathbf{w} = -\frac{1}{2}\alpha\nabla_{\mathbf{w}} J(\mathbf{w}) &= \alpha\mathbb{E}_\pi\Big[\big(q_\pi(S, A) - q_{\mathbf{w}}(S, A)\big)\Big]\nabla_{\mathbf{w}} q_{\mathbf{w}}(S, A) \\ &= \alpha\big(q_\pi(S, A) - q_{\mathbf{w}}(S, A)\big)\nabla_{\mathbf{w}} q_{\mathbf{w}}(S, A)\end{aligned}$$

Linear Action-Value Function Approximation

- Approximate with a linear combination of state-action features (action-in):

$$q_{\mathbf{w}}(s,a) = \mathbf{x}(s,a)^\top \mathbf{w} = \sum_{i=1}^{n} w_i x_i(s,a)$$

$$\Delta \mathbf{w} = \alpha\big(q_\pi(S,A) - q_{\mathbf{w}}(S,A)\big)\mathbf{x}(S,A)$$

- Approximate with a linear combination of state features (action-out):

$$q_{\mathbf{w}}(s,a) = \mathbf{x}(s)^\top \mathbf{w}_a = \sum_{i=1}^{n} w_{a,i} x_i(s)$$

$$\Delta \mathbf{w}_a = \alpha\big(q_\pi(S,A) - q_{\mathbf{w}}(S,A)\big)\mathbf{x}(S)$$

$$\Delta \mathbf{w}_{a'} = 0 \quad \text{for } a' \neq a$$

Incremental Prediction Algorithms

- Similarly to predict we substitute $q_\pi(S, A)$ for a target:

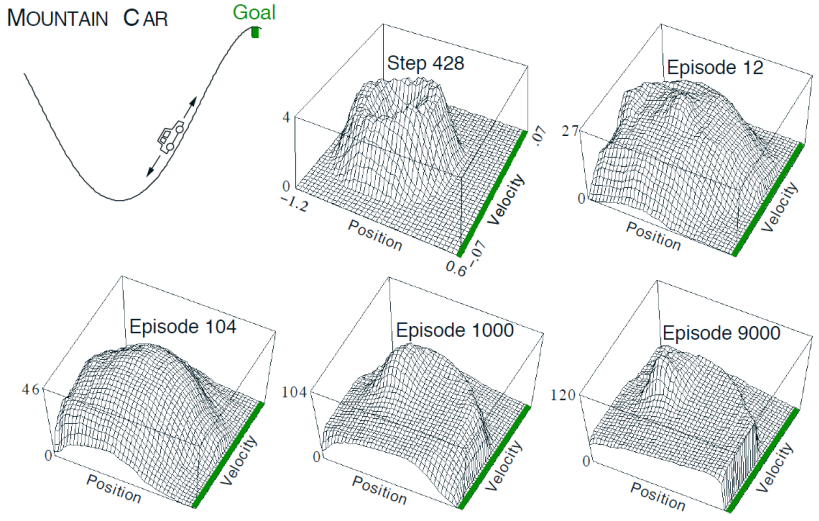
$$\Delta \mathbf{w}_t = \alpha\big(G_t - q_{\mathbf{w}}(S, A)\big)\nabla_{\mathbf{w}} q_{\mathbf{w}}(S, A) \qquad \text{(MC)}$$

$$\Delta \mathbf{w}_t = \alpha\big(R_{t+1} + \gamma q_{\mathbf{w}}(S_{t+1}, A_{t+1}) - q_{\mathbf{w}}(S, A)\big)\nabla_{\mathbf{w}} q_{\mathbf{w}}(S, A) \qquad \text{(TD(0))}$$

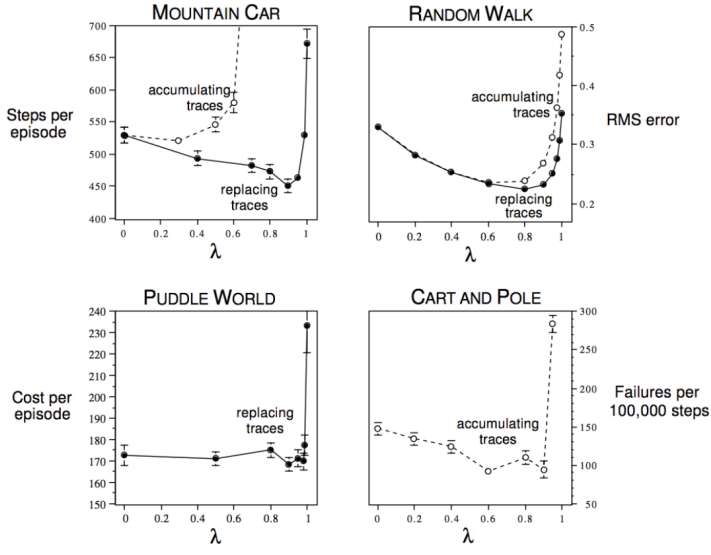
$$\Delta \mathbf{w}_t = \alpha\big(G_t^\lambda - q_{\mathbf{w}}(S, A)\big)\nabla_{\mathbf{w}} q_{\mathbf{w}}(S, A) \qquad \text{(TD}(\lambda)\text{)}$$

$$G_t^\lambda = R_{t+1} + \gamma\big((1-\lambda) q_{\mathbf{w}}(S_{t+1}, A_{t+1}) + \lambda G_{t+1}^\lambda\big)$$

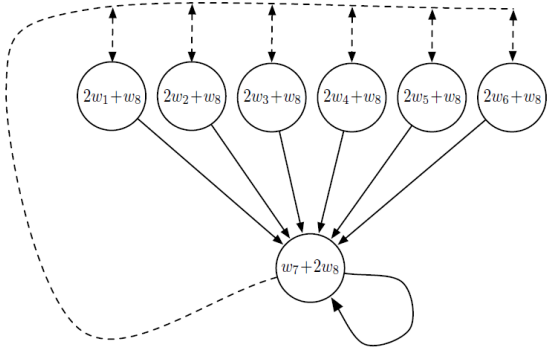
Linear SARSA with Corse Coding in Mountain Car

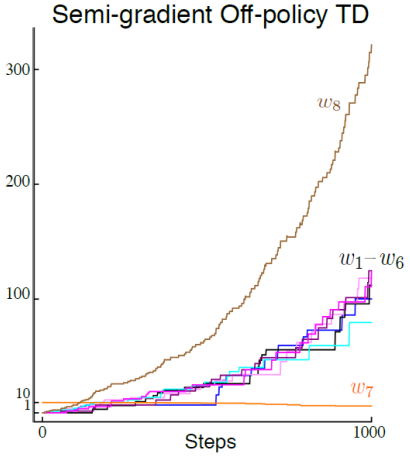


Should We Use Bootstrapping or Not?



Should We Use Bootstrapping or Not?





Monte Carlo vs. Temporal-Difference Learning

Monte Carlo

- Unbiased estimate of $v_\pi(S)$ or $q_\pi(S, A)$
- True gradient descent on $J(\mathbf{w})$
- Linear MC evaluation converges to the global optimum
- Even non-linear MC evaluation converges to a local optimum

Temporal-Difference

- Biased estimate of $v_\pi(S)$ or $q_\pi(S, A)$
- Not a true gradient descent update, because target depends on $\mathbf{w}$. That's ok, these are called semi-gradient methods.
- Linear TD evaluation converges (close) to the global optimum
- Non-linear TD evaluation can diverge.

The Deadly Triad

Algorithms that combine the following three properties can diverge:

- Function approximation
- Bootstrapping
- Off-policy learning

Batch Reinforcement Learning

- Gradient descent is simple and appealing
- But it is not sample efficient (each sample is used only once)
- Samples are correlated, which can cause instability when using nonlinear function approximators
- Batch methods seek to find the best fitting value function for a given a set of past experience (“training data")

Least-Squares Prediction

- Given a function approximator $v_{\mathbf{w}} \approx v_\pi$
- and a batch of experience $\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \ldots, \langle s_T, v_T^\pi \rangle\}$,
- we can find the best fitting value function by minimizing the mean-squared error:

$$\begin{aligned} LS(\mathbf{w}) &= \sum_{t=1}^{T} \left(v_t^\pi - v_{\mathbf{w}}(s_t)\right)^2 \\ &= \mathbb{E}_{\mathcal{D}}\left[\left(v^\pi - v_{\mathbf{w}}(s)\right)^2\right] \end{aligned}$$

- For linear function approximation, this has a closed-form solution:

$$\mathbf{w} = \left(\sum_{t=1}^{T} \mathbf{x}(s_t)\mathbf{x}(s_t)^\top\right)^{-1} \left(\sum_{t=1}^{T} v_t^\pi \mathbf{x}(s_t)\right)$$

Stochastic Gradient Descent with Experience Replay

- Given a dataset of experience

$$\mathcal{D} = \{(S_0, A_0, R_1, S_1), \ldots, (S_{T-1}, A_{T-1}, R_T, S_T)\}$$

- Repeatedly sample a random mini-batch $(s, a, r, s') \sim \mathcal{D}$ and apply an SGD update:

$$\Delta \mathbf{w} = \alpha\big(r + \gamma v_{\mathbf{w}}(s') - v_{\mathbf{w}}(s)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(s)$$

- Converges to the least-squares solution for the batch of data $\mathcal{D}$.
- Advantages:
  - ☐ Breaks temporal correlations between consecutive training samples
  - ☐ Each transition can be reused multiple times (data efficiency)
  - ☐ Can learn off-policy from data collected with any policy
  - ☐ Buffer acts as a stabilizing data distribution for training nonlinear function approximators

Deep Q-Networks (DQN)

Naive Deep Q-Learning

Replace the tabular Q-function with a neural network $Q_{\mathbf{w}}$ and apply the Q-learning update:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\big(r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a') - Q_{\mathbf{w}}(s, a)\big)\nabla_{\mathbf{w}} Q_{\mathbf{w}}(s, a)$$

Why Not Just Use Q-Learning with a Neural Network?

Applying Q-learning naively with a neural network leads to instabilities:

- Correlated data: consecutive transitions (s_t, a_t, r_t, s_{t+1}) are highly correlated
  - SGD assumes i.i.d. samples; correlations cause oscillations
- Non-stationary targets: the TD target $r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a')$ depends on $\mathbf{w}$
  - Every update changes the target – like chasing a moving goalpost
- Deadly triad: off-policy + function approximation + bootstrapping $\Rightarrow$ risk of divergence

DQN (Mnih et al., 2015) addresses these with two key stabilization techniques:

1. Experience replay
2. Fixed (frozen) target network

Fixed Q-Targets

Problem: In naive deep Q-learning both sides use the same $\mathbf{w}$:

$$\underbrace{r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a')}_{\text{target}} - \underbrace{Q_{\mathbf{w}}(s, a)}_{\text{prediction}}$$

Every parameter update shifts the target $\Rightarrow$ oscillations and divergence.

Solution: Maintain a separate target network with frozen parameters $\mathbf{w}^-$:

$$y = r + \gamma \max_{a'} Q_{\mathbf{w}^-}(s', a')$$

- $\mathbf{w}^-$ is periodically updated with the current parameters $\mathbf{w}$:

$$\mathbf{w}^- \leftarrow \tau \cdot \mathbf{w} + (1 - \tau) \cdot \mathbf{w}^-$$

- Between copies, targets are stable – more like supervised learning

Loss Function

For each mini-batch sample (s_j, a_j, r_j, s'_j) from $\mathcal{D}$, define the target:

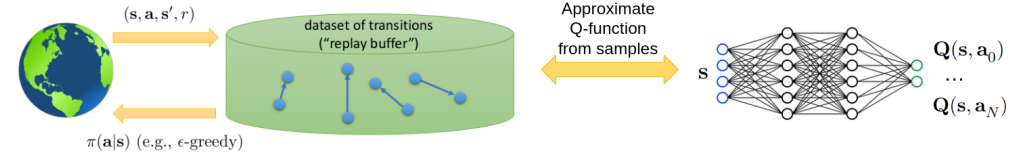
$$y_j = \begin{cases} r_j & \text{if } s'_j \text{ is terminal} \\ r_j + \gamma \max_{a'} Q_{\mathbf{w}^-}(s'_j, a') & \text{otherwise} \end{cases}$$

Minimize the mean squared TD error over the mini-batch:

$$L(\mathbf{w}) = \frac{1}{B} \sum_{j=1}^{B} \left(y_j - Q_{\mathbf{w}}(s_j, a_j) \right)^2$$

- Gradient of L is taken *only* w.r.t. the prediction $Q_{\mathbf{w}}(s_j, a_j)$
- Target y_j is treated as a constant during the gradient step
- Replay and target networks make RL look more like supervised learning

DQN Overview

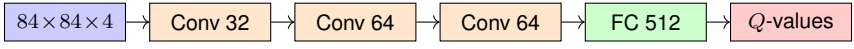


DQN Algorithm

for episode $= 1, \ldots, M$ **do**
  Observe initial state s_1
  for $t = 1, \ldots, T$ **do**
    Select action a_t using ϵ-greedy policy derived from $Q_{\mathbf{w}}$
    Execute a_t; observe reward r_{t+1} and next state s_{t+1}
    Store $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{D}$
    Sample random mini-batch $\{(s_j, a_j, r_j, s'_j)\}_{j=1}^{B}$ from $\mathcal{D}$
    Compute targets $y_j = r_j + \gamma \max_{a'} Q_{\mathbf{w}^-}(s'_j, a')$ (or $y_j = r_j$ if terminal)
    Gradient step on $\frac{1}{B} \sum_j \left(y_j - Q_{\mathbf{w}}(s_j, a_j)\right)^2$ w.r.t. $\mathbf{w}$
    Every C steps: $\mathbf{w}^- \leftarrow \tau \cdot \mathbf{w} + (1 - \tau) \cdot \mathbf{w}^-$
  end for
end for

DQN: Human-Level Control on Atari

- Environments: 49 Atari 2600 games
- Reward: game score (clipped to [-1, +1])
- Input: raw pixels (210x160 RGB)
  - Preprocess: grayscale, resize to 84x84
  - Stack 4 frames to capture motion $\Rightarrow$ input $\in \mathbb{R}^{84\times84\times4}$ (state)
- Output: Q-values for each action (action-out)
- Architecture: CNN
  - All policies use the same architecture and hyperparameters across all games



DQN: Human-Level Control on Atari

- Trained on 49 Atari 2600 games with the same architecture and hyperparams
- Only the game itself changes; same pixel input, same action output interface
- Outperformed all prior RL methods on 43 / 49 games
- Achieved human-level or better performance on 29 games
- Beat human experts on games like Breakout, Space Invaders, Pong
- First demonstration of general-purpose deep RL at scale

Mnih et al. (2015), *Human-level control through deep reinforcement learning*, *Nature*.

Beyond DQN: Key Extensions

Many improvements were proposed in the years following DQN:

Extension	Key Idea	Paper
Double DQN	Decouple selection & evaluation	van Hasselt et al., 2016
Prioritized Replay	Sample by TD error	Schaul et al., 2016
Dueling Networks	Value + advantage streams	Wang et al., 2016
Multi-step Returns	n-step TD targets	Sutton, 1988
Distributional RL	Model return distribution	Bellemare et al., 2017
Noisy Networks	Learned exploration noise	Fortunato et al., 2018
Rainbow	All of the above	Hessel et al., 2018

Double DQN

Problem: DQN's target uses the same (target) network to select and evaluate:

$$y = r + \gamma Q_{\mathbf{w}^-}\left(s', \operatorname*{argmax}_{a'} Q_{\mathbf{w}^-}(s', a')\right)$$

$\Rightarrow$ systematic overestimation bias (same issue as in tabular Q-learning).

Solution Use online network to select, target network to evaluate:

$$y = r + \gamma Q_{\mathbf{w}^-}\left(s', \operatorname*{argmax}_{a'} Q_{\mathbf{w}}(s', a')\right)$$

Prioritized Experience Replay

Problem: Uniform replay treats all transitions equally – but some are more informative.

Idea: Sample transitions with probability proportional to their TD error:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \qquad p_i = |\delta_i| + \varepsilon$$

where $\delta_i = y_i - Q_{\mathbf{w}}(s_i, a_i)$ is the TD error of transition i.

- $\alpha \in [0, 1]$: degree of prioritization ($\alpha = 0$ reduces to uniform replay)
- $\varepsilon > 0$: small constant to ensure all transitions have non-zero probability
- High TD error $\Rightarrow$ transition not yet well learned $\Rightarrow$ sample more often

Non-uniform sampling introduces bias; correct with importance sampling weights:

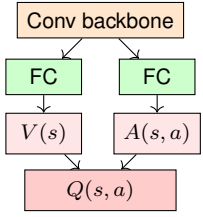
$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta, \quad \beta \text{ annealed } 0 \to 1 \text{ during training}$$

Dueling Networks

Idea Decompose $Q(s, a)$ into state value $V(s)$ and advantage $A(s, a)$:

$$Q_{\mathbf{w}}(s, a) = V_{\mathbf{w}_V}(s) + A_{\mathbf{w}_A}(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_{\mathbf{w}_A}(s, a')$$

- Shared convolutional backbone
- Two separate heads:
  - Value stream: *how good is this state?*
  - Advantage stream: *how much better is action a?*



- Subtracting mean advantage ensures identifiability
- Value stream can be updated from all actions, not just the taken one
- Especially useful when many actions have similar value

Multi-Step Returns

Recall: DQN uses a 1-step TD target:

$$y_t^{(1)} = R_{t+1} + \gamma \max_{a'} Q_{\mathbf{w}^-}(S_{t+1}, a')$$

Extension: Use n-step returns as targets:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \max_{a'} Q_{\mathbf{w}^-}(S_{t+n}, a')$$

- Intermediate n (e.g., $n = 3$) often works best
- Propagates reward signal faster through the network
- *Note*: strictly valid only for on-policy data; practical implementations accept small off-policy bias

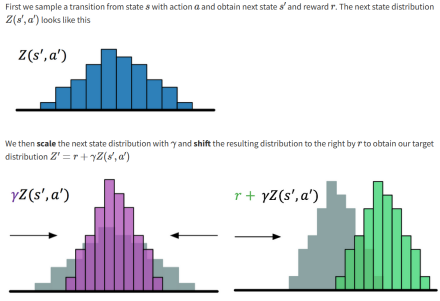
Distributional RL

Idea: Instead of estimating $\mathbb{E}[G_t]$, model the full distribution of returns $Z(s, a)$ (value distribution).

- Distributional Bellman equation:

$$Z(s, a) = R + \gamma Z(s', a')$$

- Train with cross-entropy loss between projected and predicted distributions
- Richer training signal $\Rightarrow$ better representations and performance



Noisy Networks for Exploration

Problem: ϵ-greedy adds undirected noise – same random behavior in all states.

Solution: Replace deterministic FC layers with noisy linear layers:

$$y = \left(\mu^w + \sigma^w \odot \varepsilon^w\right) x + \left(\mu^b + \sigma^b \odot \varepsilon^b\right), \qquad \varepsilon \sim \mathcal{N}(0, I)$$

- Noise parameters σ^w, σ^b are learned alongside mean parameters μ^w, μ^b
- Network can automatically reduce noise when it is confident
- Provides state-dependent exploration: explore more in uncertain regions
- Can replace ϵ-greedy entirely

Rainbow: Combining All Extensions

Rainbow integrates all six extensions into a single agent:

- New state-of-the-art on Atari at time of publication
- Ablations show all components contribute; distributional RL and prioritized replay are most important
- Rainbow reaches DQN's final performance in $7\times$ fewer environment steps

Summary

- Function approximation is essential for scaling RL to large state spaces
- Linear function approximation is simple and stable, but limited in representational power
- Deep Q-Networks (DQN) use neural networks to approximate the Q-function, enabling human-level performance on Atari games
- Key stabilizing techniques: experience replay and fixed target networks
- Many extensions to DQN have been proposed, leading to further performance improvements (e.g., Rainbow)

Next time: Policy-based and actor-critic methods for continuous control